

CyberAudit

Client Portal

Dashboard · Audit Request Form · PackSelector · Confirmation Page

Project / Projet	CyberAudit Platform
Frontend	React 19 + Vite + Tailwind CSS v4
Routing	React Router v7
Backend	Django 5.1 + DRF (REST API)
Auth	JWT via AuthContext (useAuth hook)
Key pages	DashboardPage, NewAuditPage, ConfirmationPage
Key components	ClientDashboard, AuditRequestForm, PackSelector, StatusBadge, StatCard
Source directory	src/pages/ + src/components/dashboard/ + src/components/audit/
Date	June 2026 / Juin 2026

1. Client Portal Architecture

1. Architecture du portail client

The client portal is built around three pages and five components. Each page is rendered inside a PortalLayout (with a sidebar menu). The DashboardPage acts as a router between ClientDashboard (role=client) and AdminDashboard (role=admin).

Le portail client est construit autour de trois pages et cinq composants. Chaque page est rendue dans un PortalLayout (avec menu latéral). DashboardPage sert de routeur entre ClientDashboard (role=client) et AdminDashboard (role=admin).

File / Fichier	Type	Role EN / FR
DashboardPage.jsx	Page	(EN) Routes to ClientDashboard (client) or AdminDashboard (admin) based on user.role. (FR) Route vers ClientDashboard (client) ou AdminDashboard (admin) selon user.role.
ClientDashboard.jsx	Component	(EN) Main client view: 3 StatCards + filtered requests table + notifications panel + PDF report download. (FR) Vue principale client : 3 StatCards + tableau de demandes filtré + panneau notifications + téléchargement rapport PDF.
NewAuditPage.jsx	Page	(EN) Wrapper page for the audit request form. Renders a white card with AuditRequestForm inside. (FR) Page wrapper du formulaire de demande d'audit. Affiche une carte blanche avec AuditRequestForm.
AuditRequestForm.jsx	Component	(EN) 4-field form: username, companyName, pack (PackSelector), message. Validates, calls API, sends email, navigates. (FR) Formulaire 4 champs : username, companyName, pack (PackSelector), message. Valide, appelle l'API, envoie email, navigue.
PackSelector.jsx	Component	(EN) 2-column grid: radio buttons (left) + service details panel (right). Driven by packages array from API. (FR) Grille 2 colonnes : boutons radio (gauche) + panneau services inclus (droite). Piloté par le tableau packages de l'API.
ConfirmationPage.jsx	Page	(EN) Fetches request by reference URL param. Displays case number, pack, price, estimated processing time. (FR) Récupère la demande par le paramètre d'URL référence. Affiche numéro de dossier, pack, prix, délai.
StatusBadge.jsx	Component	(EN) Pill badge for audit status: pending (amber), in_progress (blue), completed (green), archived (grey). (FR) Badge coloré pour statut: pending (amber), in_progress (bleu), completed (vert), archived (gris).
StatCard.jsx	Component	(EN) Reusable stats card with label, large value, and configurable accent color class. (FR) Carte de statistique réutilisable : libellé, valeur en grand, couleur configurable.

Route structure / Structure des routes

/dashboard	(EN) DashboardPage → ClientDashboard or AdminDashboard (role-based). (FR) DashboardPage → ClientDashboard ou AdminDashboard (selon le rôle)
/audit/new	NewAuditPage → AuditRequestForm
/audit/confirmation/:reference	(EN) ConfirmationPage : fetches request by reference param. (FR) ConfirmationPage : récupère la demande par paramètre référence

2. Dashboard : ClientDashboard

2. Tableau de bord : ClientDashboard

ClientDashboard is the main screen after login for client users. It fetches audit requests and notifications in parallel, displays live KPI cards, a filterable requests table, and a notification feed. It also handles PDF report download directly from the API.

ClientDashboard est l'écran principal après connexion pour les utilisateurs clients. Il récupère les demandes d'audit et les notifications en parallèle, affiche des cartes KPI en temps réel, un tableau de demandes filtrable, et un fil de notifications. Il gère aussi le téléchargement de rapport PDF directement depuis l'API.

State variables / Variables d'état

requests	(EN) Array of audit request objects fetched from GET /api/audits/requests/. (FR) Tableau d'objets de demande d'audit récupérés via GET /api/audits/requests/
notifications	(EN) Array of notification objects for this user, from GET /api/notifications/?user=id. (FR) Tableau de notifications pour cet utilisateur.
loading	(EN) Boolean, true while the initial data fetch is in progress. (FR) Boolean, true pendant le chargement initial.
error	(EN) Error message string or null. Shows retry button on error. (FR) Chaîne de message d'erreur ou null. Affiche un bouton Réessayer.
notice	(EN) Inline info message for report download status (202, 404, network error). (FR) Message inline pour le statut du téléchargement du rapport.
statusFilter	(EN) Selected status filter: 'all' 'pending' 'in_progress' 'completed' 'archived'. (FR) Filtre de statut sélectionné.
packFilter	(EN) Selected pack filter: 'all' 'audit' 'security' 'protection' 'premium'. (FR) Filtre de pack sélectionné.
downloading	(EN) Request ID currently downloading a PDF, or null. Disables the download button. (FR) ID de la demande en cours de téléchargement, ou null.

Data fetching : loadData() / Chargement des données

```
// ClientDashboard.jsx - parallel data fetch
const loadData = useCallback(async () => {
  setLoading(true);
  setError(null);
  try {
    const [reqs, notif] = await Promise.all([
      getAllRequests(), // GET /api/audits/requests/
      getNotificationsByUserId(user.id), // GET /api/notifications/?user=id
    ]);
  } catch (error) {
    // ...
  }
});
```

```
    });  
    setRequests(Array.isArray(reqs) ? reqs : []);  
    setNotifications(Array.isArray(notifs) ? notifs : []);  
  } catch {  
    setError("Unable to load your data. Please try again later.");  
  } finally {  
    setLoading(false);  
  }  
}, [user.id]);  
  
useEffect(() => { loadData(); }, [loadData]);
```

KPI cards : StatCard / Cartes KPI

Three StatCard components display computed values from the requests array. They are not additional API calls, they derive their values from the already-fetched requests list.

Trois composants StatCard affichent des valeurs calculées depuis le tableau requests. Ce ne sont pas des appels API supplémentaires, ils dérivent leur valeur de la liste requests déjà récupérée.

```
// ClientDashboard.jsx - KPI computation  
const openCount      = requests.filter((r) =>  
  r.status === "pending" || r.status === "in_progress").length;  
const completedCount = requests.filter((r) => r.status === "completed").length;  
  
// Rendered as:  
<StatCard label="Open requests"      value={openCount}      accentClass="text-  
amber-500" />  
<StatCard label="Completed"          value={completedCount} accentClass="text-  
green-600" />  
<StatCard label="Reports available" value={completedCount} accentClass="text-  
brand" />
```

StatCard component

```
// StatCard.jsx - props: label, value, accentClass  
export default function StatCard({ label, value, accentClass = "text-brand" }) {  
  return (  
    <div className="rounded-xl border border-gray-100 bg-white p-5 shadow-sm">  
      <p className="text-sm text-gray-500">{label}</p>  
      <p className={`mt-1 text-3xl font-extrabold ${accentClass}`}>{value}</p>  
    </div>  
  );  
}
```

Requests table with filters / Tableau des demandes avec filtres

The requests table supports two independent filters: status and pack. Filtering is done client-side on the already-fetched array using `Array.filter()`. No additional API call is made when filters change.

Le tableau des demandes supporte deux filtres indépendants : statut et pack. Le filtrage est effectué côté client sur le tableau déjà récupéré avec `Array.filter()`. Aucun appel API supplémentaire n'est fait lors du changement de filtre.

```
// ClientDashboard.jsx - client-side filter
const filteredRequests = requests.filter((r) => {
  const statusOk = statusFilter === "all" || r.status === statusFilter;
  const packOk = packFilter === "all" || r.pack?.code === packFilter;
  return statusOk && packOk;
});

// Table columns: Case # | Pack | Submission | Status | Action
// Status column uses <StatusBadge status={r.status} />
// Action column: "Download report" (completed) or "View details" (other)
```

Table columns / Colonnes du tableau

Column	Content & Source
Case #	(EN) r.reference : unique request identifier (e.g. CA-2024-0042). (FR) Identifiant unique de la demande (ex. CA-2024-0042)
Pack	(EN) r.pack?.name : nested pack object name or "—" if missing. (FR) Nom du pack imbriqué ou "—" si absent
Submission	(EN) r.submitted_at formatted with toLocaleDateString('fr-FR'). (FR) r.submitted_at formaté avec toLocaleDateString('fr-FR')
Status	(EN) <StatusBadge status={r.status} />, colored pill badge (FR) <StatusBadge status={r.status} />, badge coloré
Action	(EN) If completed: "Download report" button (triggers handleDownload). Otherwise: "View details" link to /audit/confirmation/:reference (FR) Si completed : bouton "Télécharger le rapport". Sinon : lien "View details" vers /audit/confirmation/:reference

PDF report download / Téléchargement du rapport PDF

When the user clicks "Download report", handleDownload() calls GET /api/audits/:id/report/ with responseType: 'blob'. HTTP 200 → triggers browser download. HTTP 202 → report is still being generated. HTTP 404 → no report yet.

Quand l'utilisateur clique "Télécharger le rapport", handleDownload() appelle GET /api/audits/:id/report/ avec responseType: 'blob'. HTTP 200 → déclenche le téléchargement. HTTP 202 → rapport en cours de génération. HTTP 404 → pas encore de rapport.

```
// ClientDashboard.jsx - handleDownload()
async function handleDownload(request) {
  setDownloading(request.id); // disable button
  try {
    const response = await api.get(`/audits/${request.id}/report/`, {
      responseType: "blob",
      validateStatus: (s) => s < 500, // handle 202/404 as non-errors
    });
    if (response.status === 202) { setNotice("Still generating..."); return; }
    if (response.status === 404) { setNotice("Not available yet."); return; }

    // status 200 - trigger browser download
```

```
const url = URL.createObjectURL(new Blob([response.data], { type:
"application/pdf" }));
const a = document.createElement("a");
a.href = url; a.download = `rapport-${request.reference}.pdf`;
document.body.appendChild(a); a.click();
document.body.removeChild(a); URL.revokeObjectURL(url);
} finally {
  setDownloading(null); // re-enable button
}
```

StatusBadge component

StatusBadge reads a STATUS_CONFIG lookup table and renders a small pill with a colored dot. The fallback is the 'pending' style for unknown statuses.

StatusBadge lit une table de correspondance STATUS_CONFIG et affiche un badge avec un point coloré. Le fallback est le style 'pending' pour les statuts inconnus.

Status	Badge color	Dot color	Label / Tailwind classes
pending	amber-100 bg	amber-500	"Pending" : text-amber-700 border-amber-200
in_progress	blue-100 bg	blue-500	"In Progress" : text-blue-700 border-blue-200
completed	green-100 bg	green-500	"Completed" : text-green-700 border-green-200
archived	gray-100 bg	gray-400	"Archived" : text-gray-500 border-gray-200

3. Audit Request Form : NewAuditPage + AuditRequestForm

3. Formulaire de demande d'audit : NewAuditPage + AuditRequestForm

NewAuditPage is a thin wrapper that renders the AuditRequestForm inside a white card. AuditRequestForm manages the full flow: form state, pack loading, client-side validation, API submission, email confirmation, and navigation to the confirmation page.

NewAuditPage est un wrapper léger qui affiche AuditRequestForm dans une carte blanche. AuditRequestForm gère le flux complet : état du formulaire, chargement des packs, validation côté client, soumission API, confirmation email, et navigation vers la page de confirmation.

Form fields / Champs du formulaire

Field	HTML type	Behaviour & Validation EN / FR
username	text input	(EN) Pre-filled with user.first_name. Required, max 50 chars. Pré-rempli avec user.first_name.(FR) Obligatoire, max 50 caractères.
companyName	text input	(EN) Pre-filled with user.company_name. Required, max 50 chars. Pré-rempli avec user.company_name. (FR) Obligatoire, max 50 caractères.
packCode	PackSelector	(EN) Radio-button pack selector. Required must select a pack before submission.(FR) Sélecteur de pack par boutons radio. Obligatoire un pack doit être sélectionné.
message	textarea	(EN) Optional free text, max 500 chars. Describe the audit scope. (FR) Texte libre optionnel, max 500 caractères. Décrire le périmètre d'audit.

Form state & initialisation / État du formulaire

```
// AuditRequestForm.jsx - state initialisation
const [form, setForm] = useState({
  username: user.first_name, // pre-filled from JWT user object
  companyName: user.company_name, // pre-filled from JWT user object
  packCode: "", // empty until user selects a pack
  message: "", // optional - empty by default
});

// Pack loading from API
useEffect(() => {
  async function load() {
    const packs = await getPackages(); // GET /api/audits/packages/
    setPackages(Array.isArray(packs) ? packs : (packs.results ?? []));
  }
  load();
}, []);
```


Client-side validation / Validation côté client

`validate()` checks three rules: username required + max 50, companyName required + max 50, packCode required. It returns an error map; if non-empty the form submission is stopped before any API call.

validate() vérifie trois règles : username obligatoire + max 50, companyName obligatoire + max 50, packCode obligatoire. Elle retourne une map d'erreurs ; si non vide, la soumission est stoppée avant tout appel API.

```
// AuditRequestForm.jsx - validate()
function validate() {
  const found = {};
  if (!isRequired(form.username))
    found.username = MISSING_FIELD_MESSAGE; // "Please enter the information on the form."
  else if (!isWithinMaxLength(form.username))
    found.username = `Maximum ${MAX_FIELD_LENGTH} caracteres.`;

  if (!isRequired(form.companyName))
    found.companyName = MISSING_FIELD_MESSAGE;
  else if (!isWithinMaxLength(form.companyName))
    found.companyName = `Maximum ${MAX_FIELD_LENGTH} caracteres.`;

  if (!isRequired(form.packCode))
    found.packCode = "Veuillez selectionner un pack.";

  return found; // {} means valid
}
```

Validation rules (validators.js) / Règles de validation

isRequired(value)	<code>value.trim().length > 0</code> (EN) rejects empty or whitespace-only strings (FR) Rejette les chaînes vides ou composées d'espaces
isWithinMaxLength(value)	<code>value.trim().length <= MAX_FIELD_LENGTH (50)</code> (EN) enforces character limit (FR) Force la limite de 50 caractères
MISSING_FIELD_MESSAGE	"Please enter the information on the form." (EN) exact message imposed by the spec (FR) Message exact imposé par la doc technique
MAX_FIELD_LENGTH	(EN) 50 maximum characters allowed per text field. (FR) Nombre maximum de caractères par champ texte

Submission flow / Flux de soumission

```
// AuditRequestForm.jsx - handleSubmit()
async function handleSubmit(event) {
  event.preventDefault();
  const found = validate();
  if (Object.keys(found).length > 0) { setErrors(found); return; } // stop
  setSubmitting(true);

  try {
```

```

// 1. Create audit request in Django backend
const created = await createRequest({
  packCode: form.packCode,
  message: form.message,
}); // POST /api/audits/requests/ → returns { id, reference, ... }

// 2. Fetch pack details for email
const pack = await getPackageByCode(form.packCode);

// 3. Send confirmation email via EmailJS (fire-and-forget)
sendAuditConfirmation({ to_email: user.email, ... }).catch(console.error);

// 4. Navigate to confirmation page
navigate(`/audit/confirmation/${created.reference}`);
} catch {
  setErrors({ submit: "Error sending. Please try again." });
} finally {
  setSubmitting(false);
}
}

```

Step-by-step submission / Étapes de soumission

#	Step / Étape	Details EN / FR
1	User clicks Submit	(EN) handleSubmit() called. event.preventDefault() stops browser form submission. (FR) handleSubmit() appelé. event.preventDefault() arrête la soumission native.
2	validate()	(EN) Client-side validation. If errors, setErrors(found) shows messages under fields. Returns early. (FR) Validation côté client. Si erreurs, setErrors(found) affiche les messages. Retour anticipé.
3	createRequest()	(EN) POST /api/audits/requests/ with { pack_id, scope_notes }. Returns { id, reference, status, submitted_at, ... }. (FR) POST /api/audits/requests/ avec { pack_id, scope_notes }. Retourne { id, reference, status, submitted_at, ... }.
4	getPackageByCode()	(EN) GET /api/audits/packages/:code — fetches full pack data for the email template. (FR) GET /api/audits/packages/:code — récupère les données complètes du pack pour l'email.
5	sendAuditConfirmation()	(EN) Fire-and-forget EmailJS call. Sends confirmation to user.email. Errors are caught and logged, never shown to user. (FR) Appel EmailJS fire-and-forget. Erreurs captées et loguées, jamais affichées.
6	navigate()	(EN) navigate('/audit/confirmation/:reference'). redirects to ConfirmationPage. (FR) Redirige vers ConfirmationPage.

4. PackSelector Component

4. Composant PackSelector

PackSelector is a controlled component receiving three props from AuditRequestForm: the packages array loaded from the API, the currently selected code, and the onSelect callback. It renders a two-column layout: radio buttons on the left, service details on the right.

PackSelector est un composant contrôlé qui reçoit trois props de AuditRequestForm : le tableau packages chargé depuis l'API, le code actuellement sélectionné, et le callback onSelect. Il affiche une mise en page à deux colonnes : boutons radio à gauche, détails des services à droite.

Props

packages	(EN) Array<Pack> list fetched from GET /api/audits/packages/. Each pack: { code, name, included_services, for_whom, perimeter, duration_days, price }. (FR) Tableau chargé depuis l'API. Chaque pack : { code, name, included_services, for_whom, perimeter, duration_days, price }.
selectedCode	(EN) String the code of the currently selected pack (or "" if none). (FR) Code du pack actuellement sélectionné (ou "" si aucun).
onSelect(code)	(EN) Callback fired when a radio button is clicked. Updates form.packCode in the parent. (FR) Callback déclenché au clic sur un bouton radio. Met à jour form.packCode dans le parent.

Layout 2 column grid / Mise en page grille 2 colonnes

```
// PackSelector.jsx - 2-column responsive grid
const selectedPack = packages.find(pack => pack.code === selectedCode) || null;

return (
  <div className="grid gap-5 sm:grid-cols-2">

    {/* Left column: radio buttons */}
    <div>
      <p className="mb-2 text-sm font-semibold text-gray-700">Select pack:</p>
      <div className="flex flex-col gap-2">
        {packages.map(pack => (
          <label key={pack.code} className={`flex cursor-pointer items-center gap-
2
          rounded-md border px-3 py-2 text-sm transition
          ${selectedCode === pack.code
            ? "border-brand bg-brand-soft" // selected: blue highlight
            : "border-gray-200 hover:border-brand/50"}`, // unselected
          >
            <input type="radio" name="pack" value={pack.code}
              checked={selectedCode === pack.code}
              onChange={() => onSelect(pack.code)} className="accent-brand" />
            <span className="font-medium text-gray-800">{pack.name}</span>
          </label>
        ))}
      </div>
    </div>
```

```

</div>

{ /* Right column: service detail panel */
<div className="rounded-md border border-gray-200 bg-cream p-4 text-sm">
  {!selectedPack
    ? <p className="text-gray-400">Select a package to view service
details.</p>
    : <dl>
      <dt>Included services</dt><dd>{selectedPack.included_services}</dd>
      <dt>For whom</dt><dd>{selectedPack.for_whom}</dd>
      <dt>Perimeter</dt><dd>{selectedPack.perimeter}</dd>
      <dt>Analysis estimate</dt><dd>{selectedPack.duration_days} business
days</dd>
      <dt>Price</dt><dd className="font-bold">{selectedPack.price} EUR</dd>
    </dl>
  }
</div>
</div>
);

```

Service detail panel fields / Champs du panneau de détail

Field (Pack model)	Description EN / FR
included_services	(EN) List of services provided in this pack (e.g. 'Penetration test, Vulnerability scan'). (FR) Liste des services fournis dans ce pack (ex. 'Test d'intrusion, Scan de vulnérabilités').
for_whom	(EN) Target audience description (e.g. 'SMEs with 10-50 employees'). (FR) Description du public cible (ex. 'PME de 10-50 employés').
perimeter	(EN) Analysis scope (e.g. 'Web application + internal network'). Périètre d'analyse (ex. 'Application web + réseau interne').
duration_days	(EN) Estimated processing time in business days (integer). (FR) Délai estimé de traitement en jours ouvrés (entier).
price	(EN) Pack price in EUR (displayed as "X EUR" in both PackSelector and ConfirmationPage)..(FR) Prix du pack en EUR (affiché "X EUR" dans PackSelector et ConfirmationPage).

Responsive behaviour / Comportement responsive

On mobile (< sm breakpoint = 640px), the grid is 1 column: radio buttons appear above the service panel. On desktop (>= sm), both columns sit side-by-side. The layout uses Tailwind's sm:grid-cols-2 utility class.

Sur mobile (< sm = 640px), la grille est à 1 colonne : les boutons radio apparaissent au-dessus du panneau de services. Sur bureau (>= sm), les deux colonnes sont côte à côte. La mise en page utilise la classe utilitaire Tailwind sm:grid-cols-2.

5. Confirmation Page : ConfirmationPage

5. Page de confirmation : ConfirmationPage

After form submission, the user is navigated to `/audit/confirmation/:reference`. ConfirmationPage reads the reference URL parameter, fetches the full request object from the API, and displays a complete summary card with a green success header.

Après la soumission du formulaire, l'utilisateur est redirigé vers `/audit/confirmation/:reference`. ConfirmationPage lit le paramètre d'URL référence, récupère l'objet de demande complet depuis l'API, et affiche une carte de résumé complète avec un en-tête de succès vert.

URL parameter & data fetch / Paramètre URL et chargement

```
// ConfirmationPage.jsx - URL param + data fetch
const { reference } = useParams(); // e.g. "CA-2024-0042" from URL

const [request, setRequest] = useState(null);
const [loading, setLoading] = useState(true);

useEffect(() => {
  async function load() {
    const data = await getRequestByReference(reference);
    // GET /api/audits/requests/?reference=CA-2024-0042
    setRequest(data);
    setLoading(false);
  }
  load();
}, [reference]);
```

Confirmation card fields / Champs de la carte de confirmation

Field / Champ	Source & Notes EN / FR
Case number	(EN) request.reference, unique reference generated by the Django backend. (FR) Référence unique générée par le backend Django.
Contact	(EN) clientInfo.first_name + clientInfo.last_name (sanitize() applied) (FR) sanitize() appliqué aux noms pour prévenir XSS.
Company Name	(EN) clientInfo.company_name or "—" if empty. (FR) clientInfo.company_name ou "—" si absent.
Selected pack	(EN) pack.name, from the nested pack object. (FR) pack.name depuis l'objet pack imbriqué.
Services included	(EN) pack.included_services + pack.for_whom + pack.perimeter (conditional: shown only if included_services is present). (FR) Affiché uniquement si included_services est présent.
Price	(EN) pack.price + " EUR" displayed in bold. (FR) pack.price + " EUR" affiché en gras.

Submission date	request.submitted_at formatted as "DD/MM/YYYY HH:MM" (fr-FR locale). Request.submitted_at formaté en "JJ/MM/AAAA HH:MM" (locale fr-FR).
Estimated processing	(EN) pack.duration_days + " business days» derived from selected pack. (FR) pack.duration_days + " jours ouvrés» dérivé du pack sélectionné.
Message	request.scope_notes (sanitize() applied) or "—" if empty. request.scope_notes (sanitize() appliqué) ou "—" si absent.

sanitize() : XSS protection / Protection XSS

Every user-supplied text field is passed through sanitize() before rendering. This prevents Cross-Site Scripting by escaping HTML special characters in the output. The function is used on reference, names, company, scope_notes, and pack name.

Chaque champ texte fourni par l'utilisateur passe par sanitize() avant le rendu. Cela prévient le Cross-Site Scripting en échappant les caractères HTML spéciaux. La fonction est utilisée sur référence, noms, société, scope_notes et nom du pack.

State handling & navigation / Gestion des états et navigation

Loading state	(EN) Renders a centered animated spinner (animate-spin). No content is shown while fetching. (FR) Affiche un spinner centré animé. Aucun contenu pendant le chargement.
Not found state	(EN) If request is null after fetch, shows "No requests found for: reference" + link to /dashboard. (FR) Si request est null après le fetch, affiche un message + lien vers /dashboard.
Success state	(EN) CheckCircle2 icon (Lucide) + green heading + summary card + amber info message + two action buttons. (FR) Icone CheckCircle2 + titre vert + carte récap + message amber + deux boutons.
"Go to Dashboard" button	(EN) navigate('/dashboard') returns to the requests list. (FR) Retour vers la liste des demandes.
"Submit a new request" button	(EN) navigate('/audit/new') starts a fresh audit request form. (FR) Démarre un nouveau formulaire de demande d'audit.

formatDateTime() helper

```
// ConfirmationPage.jsx - date formatting
function formatDateTime(isoString) {
  const d = new Date(isoString);
  return `${d.toLocaleDateString("fr-FR")} ${
    d.toLocaleTimeString("fr-FR", { hour: "2-digit", minute: "2-digit" })
  }`;
  // Output example: "17/06/2026 14:23"
}
```

6. Summary

6. Résumé

Concept	Implementation	Notes EN / FR
Dashboard routing	DashboardPage.jsx	user.role === 'client' → ClientDashboard; else → AdminDashboard
KPI cards	StatCard × 3	(EN) Open requests (amber), Completed (green), Reports (brand). Derived from requests array, no extra API call. (FR) Demandes ouvertes (ambre), Terminées (vert), Rapports (marque). Dérivé du tableau de demandes, pas d'appel API supplémentaire.
Requests table	ClientDashboard	(EN) 5 columns. Client-side filter by status + pack. StatusBadge for status column. (FR) 5 colonnes. Filtre côté client par statut + pack. BadgeStatut pour la colonne statut.
PDF download	handleDownload()	(EN) responseType: 'blob'. HTTP 202 → still generating. HTTP 404 → not ready. HTTP 200 → browser download trigger. (FR) responseType : 'blob'. HTTP 202 → encore en cours de génération. HTTP 404 → pas encore prêt. HTTP 200 → déclenche le téléchargement dans le navigateur.
Status badge	StatusBadge	(EN) 4 statuses: pending (amber), in_progress (blue), completed (green), archived (grey). (FR) 4 statuts : en attente (ambre), en cours (bleu), terminé (vert), archivé (gris).
Audit form	AuditRequestForm	(EN) 4 fields. username + companyName pre-filled. PackSelector required. Validates before API call. (FR) 4 champs. Nom d'utilisateur + nom de l'entreprise pré-remplis. Sélecteur de pack requis. Vérifie avant l'appel API.
Pack selection	PackSelector	(EN) 2-column grid. Radio buttons + service details panel. Responsive (sm:grid-cols-2). (FR) Grille à 2 colonnes. Boutons radio + panneau de détails du service. Réactif (sm:grid-cols-2).
Submission flow	handleSubmit()	(EN) validate → createRequest → getPackageByCode → sendAuditConfirmation (fire-and-forget) → navigate. (FR) valider → createRequest → getPackageByCode → sendAuditConfirmation (envoyer-et-oublier) → naviguer.
Confirmation	ConfirmationPage	(En) Fetches by reference URL param. 10 fields. sanitize() on all user text. Links back to dashboard. (FR) Récupère par paramètre URL de référence. 10 champs. sanitize() sur tout le texte

		utilisateur. Fait un lien retour vers le tableau de bord.
XSS protection	sanitize()	(EN) Applied to all user-supplied text before React render. Escapes HTML special characters. (FR) Récupère par paramètre URL de référence. 10 champs. sanitize() sur tout le texte utilisateur. Fait un lien retour vers le tableau de bord.

Data flow: Django REST API → dataService.js (axios via api.js) → React state (useState) → filtered display. All text from the API goes through sanitize() before rendering. No component calls the API directly, all API access,s is routed through dataService.js functions.

Flux de données : API Django REST → dataService.js (axios via api.js) → état React (useState) → affichage filtré. Tout texte provenant de l'API passe par sanitize() avant le rendu. Aucun composant n'appelle l'API directement tout accès API est routé via les fonctions de dataService.js.